API GUIDE

Beginner to Advanced • API Fundamentals, REST, HTTP, Java & Spring Boot

A practical learning handbook with concepts, examples, endpoint design, security, testing, and interview preparation.

PART 1	API basics, types, request/response, HTTP methods and status codes
PART 2	REST API design, JSON, parameters, CRUD and pagination
PART 3	Build APIs with Java and Spring Boot
PART 4	Validation, error handling, security, JWT and CORS
PART 5	Testing, documentation, microservices and deployment
PART 6	Practice project, checklist and interview questions

Designed especially for learners working with Java, Spring Boot, REST APIs, and backend projects.

How to Use This Guide

Read the sections in order if you are new to APIs. If you already know HTTP and REST, jump to the Spring Boot section and use the endpoint checklist and practice project to build confidence.

Learning path

- Understand what an API is and why applications use APIs.
- Learn HTTP requests, responses, methods, headers, status codes, and JSON.
- Design REST endpoints around resources and implement CRUD operations.
- Build the same API in Spring Boot using Controller → Service → Repository layers.
- Add validation, consistent errors, authentication, authorization, and tests.
- Document the API, consider production concerns, and practice interview questions.

Important scope note

“API” is a broad term. This guide focuses on the most useful backend API skills for Java/Spring Boot developers, especially HTTP and REST APIs. It also introduces other API styles so you can recognize them.

1. What Is an API?

API stands for **Application Programming Interface**. It is a defined way for one software component to request data or an action from another component. Think of it as a menu and ordering rules: the client asks for something in an agreed format, and the server responds in an agreed format.

Simple example

A weather app does not need to operate weather satellites itself. It sends a request to a weather service API, receives a response such as temperature and conditions, and displays the data.

```
Client (browser/mobile app)
        |
        | HTTP request: GET /api/weather?city=Patna
        v
Backend API server
        |
        | obtains data / applies business rules
        v
HTTP response: 200 OK + JSON
```

Common API participants

- **Client:** browser, mobile app, another backend, command-line tool, or automated job.
- **Server:** application that receives requests and returns responses.
- **Endpoint:** a URL/path plus an HTTP method, such as GET /api/cars.
- **Contract:** the documented request, response, authentication, and error rules.
- **Payload:** data sent in the request or response, often JSON.

2. Common Types of APIs

Type	How it works	Typical use
REST	Resources over HTTP; often JSON	Web/mobile backends, CRUD
SOAP	Structured XML messages with formal contracts	Some enterprise/legacy systems

Type	How it works	Typical use
GraphQL	Client asks for specific fields through a query schema	Flexible data fetching
WebSocket	Persistent two-way connection	Chat, live dashboards, multiplayer
gRPC	Remote procedure calls, commonly Protocol Buffers over HTTP/2	Efficient service-to-service communication
Library/API	Functions/classes exposed by a package	Calling Java methods in a library

For most beginner Java backend roles, start with HTTP + REST + JSON. Learn the other styles at a high level, then go deeper when a project requires them.

3. HTTP Request and Response

A request contains

- **Method:** what the client wants to do (GET, POST, PUT, PATCH, DELETE).
- **Path:** which resource is involved, e.g. /api/cars/42.
- **Headers:** metadata, e.g. Content-Type, Accept, Authorization.
- **Query parameters:** optional filters or pagination, e.g. ?page=0&size;=10.
- **Body:** submitted data, commonly JSON; usually used with POST, PUT, or PATCH.

Example request

```
POST /api/cars HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer <access-token>

{
  "brand": "Toyota",
  "model": "Corolla",
  "dailyRate": 2500
}
```

Example response

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/cars/42

{
  "id": 42,
  "brand": "Toyota",
  "model": "Corolla",
  "dailyRate": 2500
}
```

Important headers

- **Content-Type:** format of the body being sent, e.g. application/json.
- **Accept:** formats the client can receive, e.g. application/json.
- **Authorization:** credentials or access token.
- **Cache-Control:** caching instructions.
- **Location:** often points to a newly created resource after 201 Created.

4. HTTP Methods and CRUD

Method	Typical meaning	Example	Common success status
GET	Read resource(s)	GET /api/cars	200 OK
POST	Create resource / submit action	POST /api/cars	201 Created
PUT	Replace resource representation	PUT /api/cars/42	200 or 204
PATCH	Partially update resource	PATCH /api/cars/42	200 or 204
DELETE	Delete resource	DELETE /api/cars/42	204 or 200

CRUD means Create, Read, Update, Delete. It is a useful mapping, not a rule that every API must have all four operations.

GET should be safe: it should not intentionally change server state. GET, PUT, and DELETE are generally idempotent by HTTP semantics; repeating the same request should have the same intended effect. POST is not generally idempotent.

5. HTTP Status Codes

Code	Meaning	Use it when
200 OK	Request succeeded	Returning requested data or successful update
201 Created	A resource was created	POST creates a resource
202 Accepted	Accepted for later processing	A background job has been queued
204 No Content	Success, no response body	Delete or update succeeded without a body
400 Bad Request	Malformed or invalid request	Bad syntax or invalid request structure
401 Unauthorized	Authentication is missing/invalid	Client must authenticate
403 Forbidden	Authenticated but not allowed	User lacks permission
404 Not Found	Resource does not exist	Requested ID/path cannot be found
409 Conflict	Conflicts with current state	Duplicate unique value or booking conflict
422 Unprocessable Content	Understood but semantically invalid	Often used for validation failures
429 Too Many Requests	Rate limit exceeded	Client sends too many requests
500 Internal Server Error	Unexpected server failure	Unhandled server-side error
503 Service Unavailable	Temporarily unable to serve	Maintenance or overloaded dependency

Avoid returning 200 for every outcome. Correct status codes make APIs easier to use, debug, monitor, and test.

6. JSON, Parameters and API Design

JSON example

```
{
  "id": 42,
  "brand": "Toyota",
  "available": true,
  "features": ["Bluetooth", "Backup camera"],
  "owner": null
}
```

JSON supports objects, arrays, strings, numbers, booleans, and null. Use consistent field names and predictable types.

Parameter types

- **Path variable:** identifies a specific resource — /api/cars/42.

- **Query parameter:** filters, sorts, searches, or paginates — /api/cars?brand=Toyota.
- **Request body:** structured data sent to create or update — JSON in POST /api/cars.
- **Header:** metadata such as authorization or content negotiation.

Good endpoint design

- Prefer nouns/resources: /api/cars, /api/bookings, /api/users.
- Use HTTP methods to express the operation rather than action-heavy paths.
- Keep naming consistent; choose singular/plural conventions and stick to them.
- Use IDs to identify individual resources: /api/cars/{id}.
- Support pagination for large collections: ?page=0&size;=20.
- Use filtering and sorting when useful: ?status=ACTIVE&sort;=createdAt,desc.
- Version deliberately when breaking changes are required, e.g. /api/v1/cars.
- Do not expose internal database details or stack traces in public responses.

Pagination response example

```
{
  "content": [
    {"id": 42, "brand": "Toyota"},
    {"id": 43, "brand": "Honda"}
  ],
  "page": 0,
  "size": 20,
  "totalElements": 73,
  "totalPages": 4
}
```

7. Build a REST API with Java and Spring Boot

A common Spring Boot structure separates HTTP concerns, business logic, and database access. This makes the application easier to test and maintain.

```
Client
  -> Controller (@RestController): HTTP routing and input/output
  -> Service (@Service): business rules
  -> Repository (@Repository / Spring Data): database access
  -> Database
```

Example entity

```
@Entity
public class Car {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String brand;
    private String model;
    private BigDecimal dailyRate;

    // getters and setters
}
```

Repository

```
public interface CarRepository extends JpaRepository<Car, Long> {
    List<Car> findByBrandIgnoreCase(String brand);
}
```

Service

```
@Service
```

```
public class CarService {
    private final CarRepository repository;

    public CarService(CarRepository repository) {
        this.repository = repository;
    }

    public List<Car> findAll() {
        return repository.findAll();
    }

    public Car findById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new CarNotFoundException(id));
    }

    public Car create(Car car) {
        return repository.save(car);
    }
}
```

Controller

```
@RestController
@RequestMapping("/api/cars")
public class CarController {
    private final CarService service;

    public CarController(CarService service) {
        this.service = service;
    }

    @GetMapping
    public List<Car> getCars() {
        return service.findAll();
    }

    @GetMapping("/{id}")
    public Car getCar(@PathVariable Long id) {
        return service.findById(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Car createCar(@RequestBody Car car) {
        return service.create(car);
    }
}
```

This is a teaching example. A production API should normally use request/response DTOs, validation, consistent error handling, and explicit rules for which fields clients are allowed to set.

Key Spring annotations

Annotation	Purpose
@RestController	Declares a controller whose return values are written to the response body
@RequestMapping	Defines a shared path or request mapping
@GetMapping	@PostMapping, @PutMapping, @PatchMapping, @DeleteMapping Maps HTTP methods to Java methods
@PathVariable	Reads a value from the URL path
@RequestParam	Reads a query parameter
@RequestBody	Converts the request body (often JSON) into a Java object
@Service	Marks a service-layer component
@Repository	Marks a persistence component; Spring Data repositories are often interfaces

Annotation	Purpose
@Valid	Triggers bean validation on supported request objects
@ExceptionHandler	Handles selected exceptions in a controller/advice
@RestControllerAdvice	Shares exception handling across controllers

8. DTOs, Validation and Error Handling

Why use DTOs?

A Data Transfer Object (DTO) defines the fields the API accepts or returns. It prevents accidental exposure of internal entity fields and helps keep the public contract stable even when database models change.

```
public record CreateCarRequest(
    @NotBlank String brand,
    @NotBlank String model,
    @NotNull @Positive BigDecimal dailyRate
) {}
```

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public CarResponse create(@Valid @RequestBody CreateCarRequest request) {
    return service.create(request);
}
```

Validation annotations require the appropriate validation dependency and validation support in the project.

Consistent error response

```
{
  "timestamp": "2026-10-24T10:15:30Z",
  "status": 404,
  "error": "Not Found",
  "message": "Car with id 42 was not found",
  "path": "/api/cars/42"
}
```

- Use a shared exception handler, commonly with @RestControllerAdvice.
- Return safe, actionable messages to clients; log technical details server-side.
- Never expose passwords, access tokens, database credentials, or stack traces in error responses.
- Choose status codes consistently and document expected error cases.

9. Authentication and Authorization

Authentication asks “Who are you?” **Authorization** asks “What are you allowed to do?” They are related but different.

Common approaches

- **Session + cookie:** the server stores session state; common for traditional web applications.
- **API key:** a key identifies an application or client; do not treat every API key as a user identity.
- **Bearer token:** the client sends a token in the Authorization header.
- **OAuth 2.0 / OpenID Connect:** common standards for delegated authorization and user identity.
- **JWT:** a token format often used to carry signed claims. A JWT is not automatically encrypted, and its claims must be validated.

Bearer token request

```
GET /api/bookings/me HTTP/1.1
```

```
Authorization: Bearer <access-token>
Accept: application/json
```

Security checklist

- Use HTTPS in production.
- Validate authentication tokens and their signature, issuer, audience, and expiration as appropriate.
- Check permissions on every protected operation; do not rely only on hiding buttons in the frontend.
- Use least privilege and avoid hard-coded secrets.
- Validate input and limit request sizes.
- Apply rate limiting where abuse is possible.
- Use secure password hashing; never store plain-text passwords.
- Configure CORS narrowly; CORS is not an authentication mechanism.
- Avoid logging credentials, authorization headers, or sensitive personal data.

10. CORS, Cookies and CSRF

CORS is a browser security mechanism controlling whether a web page from one origin may read responses from another origin. Configure allowed origins, methods, and headers intentionally.

CSRF is a risk when browsers automatically attach credentials (such as session cookies) to requests. Protection depends on the authentication approach; cookie-based applications commonly need CSRF defenses. Stateless bearer-token APIs have a different threat model, but token storage and XSS still matter.

Do not solve a CORS error by allowing every origin in production without understanding the consequences.

11. Testing APIs

Testing layers

- **Unit tests:** test business logic in a small, isolated scope.
- **Controller tests:** verify routing, request binding, validation, status codes, and response bodies.
- **Integration tests:** verify interactions among Spring components and infrastructure such as the database.
- **Contract tests:** verify that consumers and providers agree on the API contract.
- **End-to-end tests:** exercise a complete flow through the running application.

Example test cases for POST /api/cars

- Valid payload creates a car and returns 201 Created.
- Missing required brand returns a validation error (commonly 400).
- Negative dailyRate is rejected.
- Response contains the expected ID and fields.
- Unauthenticated request is rejected when authentication is required.
- A user without permission cannot create or modify protected data.

Tools

Common choices include curl, Postman, Bruno, Spring MockMvc, JUnit, Mockito, and Testcontainers. Use the tools appropriate to your project and test layer.

12. API Documentation

Documentation should tell a consumer how to call an endpoint without reading your source code. OpenAPI is a widely used specification for describing HTTP APIs; Swagger UI can present an interactive view when configured.

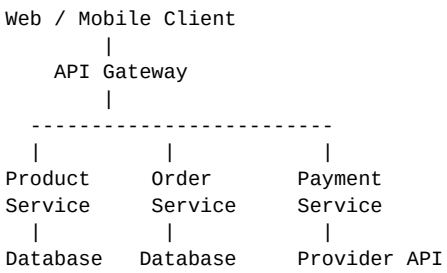
Document each endpoint

- Method and path.
- Purpose and required permissions.
- Path/query parameters, headers, and request schema.
- Success response schema and examples.
- Possible error statuses and error body format.
- Pagination, sorting, filtering, and rate limits.
- Idempotency behavior for operations where retries matter.

Keep examples aligned with actual application behavior. Outdated documentation is often worse than no documentation.

13. Microservices and External APIs

In a microservice system, each service owns a focused capability and may expose an API to other services. For example, an e-commerce system may contain Product, Order, Payment, and Notification services.



Important concepts

- **API Gateway:** common entry point for routing, authentication, rate limits, and cross-cutting policies.
- **Service discovery:** helps services locate each other in dynamic environments.
- **Timeouts and retries:** prevent calls from waiting forever; retries must be safe and bounded.
- **Circuit breaker:** stops repeatedly calling an unhealthy dependency for a period.
- **Idempotency key:** can prevent duplicate effects when a client retries operations such as payments.
- **Events and messaging:** asynchronous communication can reduce direct coupling but adds eventual-consistency concerns.
- **Observability:** logs, metrics, traces, correlation IDs, and useful health checks.

Do not split a simple application into microservices only because the architecture is popular. Distributed systems introduce network failures, deployment complexity, and data-consistency challenges.

14. Performance and Production Readiness

- Paginate large list endpoints; avoid returning unbounded collections.
- Use database indexes based on real query patterns and inspect slow queries.
- Set connection, read, and overall request timeouts.

- Cache only where freshness and invalidation rules are understood.
- Use rate limits and request-size limits to protect capacity.
- Avoid N+1 database queries; inspect ORM-generated SQL.
- Use structured logs and correlation/request IDs.
- Monitor latency percentiles, error rate, traffic, and saturation.
- Keep secrets in environment-specific secret management, not source control.
- Use backward-compatible changes where possible and version breaking changes deliberately.

15. Example Practice API: Car Rental

Build this small API to practice resources, CRUD, validation, authentication, and testing. It is a sample contract, not a claim about an existing repository.

Method	Endpoint	Purpose
GET	/api/cars	List cars with pagination/filtering
GET	/api/cars/{id}	Get one car
POST	/api/cars	Create a car (admin)
PATCH	/api/cars/{id}	Update selected car fields (admin)
DELETE	/api/cars/{id}	Delete/archive a car (admin)
GET	/api/cars/{id}/availability?from=...&to=...	Check availability
POST	/api/bookings	Create a booking
GET	/api/bookings/me	List current user's bookings
DELETE	/api/bookings/{id}	Cancel a permitted booking

Practice sequence: (1) implement GET endpoints, (2) add POST and validation, (3) add update/delete, (4) connect a database, (5) add authentication and authorization, (6) write tests, (7) document with OpenAPI, (8) containerize and deploy.

16. API Design Checklist

- ■ Endpoint names describe resources and follow a consistent convention.
- ■ HTTP methods and status codes match the operation.
- ■ Request/response schemas are explicit and stable.
- ■ Input validation is implemented on the server.
- ■ Errors use a consistent, safe response structure.
- ■ Authentication and authorization are checked where needed.
- ■ Sensitive values are not returned or logged.
- ■ List endpoints support pagination where appropriate.
- ■ Duplicate submissions and retry behavior are considered.
- ■ Tests cover success, invalid input, missing resources, and access control.
- ■ API documentation and examples are current.
- ■ Timeouts, logging, monitoring, and rate limits are considered for production.

17. Interview Questions with Short Answers

Q: What is an API?
A: A defined interface through which software components communicate.

Q: What is REST?
A: An architectural style commonly using HTTP methods and resource-oriented URLs.

Q: What is the difference between PUT and PATCH?
A: PUT generally replaces a resource representation; PATCH applies a partial modification.

Q: What is the difference between 401 and 403?
A: 401 means authentication is missing or invalid; 403 means the request is not permitted.

Q: What is idempotency?
A: Repeating an operation has the same intended effect as performing it once.

Q: What is a DTO?
A: An object that defines data transferred into or out of an application/API.

Q: Why use a service layer?
A: To isolate business logic from HTTP/controller and persistence concerns.

Q: What is CORS?
A: A browser-enforced policy controlling cross-origin access to responses.

Q: What is JWT?
A: A token format that can carry claims; tokens must be validated and are not necessarily encrypted.

Q: How do you secure an API?
A: Use HTTPS, strong authentication, authorization checks, validation, safe secret handling, rate limits, and appropriate logging.

Q: What is API versioning?
A: A strategy for managing changes to a public contract while controlling compatibility.

Q: How do you test a Spring REST API?
A: Use unit tests, MockMvc/controller tests, integration tests, and end-to-end tests as appropriate.

18. Suggested Learning Order

- **Stage 1:** API basics, HTTP methods, headers, status codes, JSON.
- **Stage 2:** REST endpoint design, CRUD, query/path parameters, pagination.
- **Stage 3:** Spring Boot controllers, services, repositories, DTOs, JPA.
- **Stage 4:** validation, exception handling, JUnit, Mockito, MockMvc.
- **Stage 5:** Spring Security, authentication, authorization, JWT/OAuth concepts.
- **Stage 6:** OpenAPI documentation, integration testing, Docker, deployment.
- **Stage 7:** microservices, gateway, messaging, resilience, observability.

Glossary

Term	Meaning
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol

Term	Meaning
JSON	JavaScript Object Notation
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
JWT	JSON Web Token
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
ORM	Object-Relational Mapping
OpenAPI	Specification for describing HTTP APIs
SLA/SLO	Service Level Agreement / Service Level Objective

End of guide. Keep learning by building a small API, testing its edge cases, and reading the API contracts of real projects.